

Hospital billing integrated with HIS

Prototype build, ER model, API, local setup and AWS classroom demo guide

7 October 2026 | Source: <https://github.com/sreehariX/tiveri-billing>

This guide accompanies the 21-slide final presentation and the working Next.js, FastAPI and PostgreSQL prototype. Every patient, tariff, diagnosis, payer response and transaction is invented.

1. Scope and assumptions

The working scope represents a 20 to 50 bed Indian hospital. Doctor, pharmacy and billing staff sign in to one shared encounter. The API also models OPD, IPD, emergency and day care settings, along with self pay, private TPA, PM-JAY, CGHS and corporate payer routes.

- + Doctor: records a clinical note, provisional ICD-10, delivered service and prescription.
- + Pharmacy: checks a prescription and unexpired batch, then posts stock movement and a charge.
- + Billing: reviews the running bill, room stay and payer status, generates an itemised invoice in place, and records a receipt.
- + Private TPA and PM-JAY decisions are local simulations. No official portal credentials or real patient data are used.

The three classroom accounts are admin, doctor and pharmacy. Each uses the training password Demo@1234. Production identity and privacy controls are outside this teaching prototype.

2. Architecture and data flow

Next.js serves the interface on port 3000 and forwards same-origin /api requests to FastAPI on port 8000. FastAPI validates roles, encounter status, stock and payer conditions. PostgreSQL stores the linked records and audit events. Money is stored as integer paise.

Layer	Responsibility
HIS side	Patient and encounter, doctor notes, prescriptions, delivered service events.
Billing side	Charge and tax snapshots, running bill, final invoice, claim and receipt.
Integration boundary	Unique source_event_id makes a repeated HIS event idempotent. A changed retry is rejected.

The presentation includes the student's supplied Chen notation ER diagram. It shows a broader HIS model with LIS and RIS. Those modules are conceptual context; the live site implements selected care, pharmacy and billing paths.

3. Database and API

Record	Key link or rule
patient to encounter	One patient may have several encounters. Payer route belongs to the encounter.
encounter to charge	Each delivered event or dispense adds a charge. Source event IDs are unique.
encounter to invoice	At most one final invoice. Each invoice line preserves its price and tax snapshot.
invoice to claim / receipt	Approval and money received are stored separately.
prescription to dispense	A stock batch must be unexpired and have enough units.

The schema is in `backend/app/schema.sql` and the FastAPI implementation is in `backend/app/main.py`. The API contract is in `docs/api_contract.md`. Open <http://127.0.0.1:8000/docs> for request and response schemas.

POST /api/his/events	delivered service
POST /api/clinical/notes	doctor note and ICD-10
POST /api/clinical/prescriptions	
POST /api/pharmacy/dispense	stock and charge together
POST /api/invoices	one final itemised invoice
POST /api/preauth	simulated payer request
POST /api/claims	simulated claim
POST /api/receipts	money received
GET /api/gstr1	monthly review packet

4. Indian billing rules shown in the demo

The 2025 GST Council FAQ gives a general 5% rate for drugs and medicines except specified nil-rated entries. The CBIC schedule lists contraceptive products under HSN 3006 at nil rate. The prototype uses generic training items, not a verified branded medicine catalog. Separate OPD pharmacy sales follow each item's configured rate.

A medicine supplied as part of qualifying inpatient healthcare is modelled as an exempt composite care supply. The specified non-ICU room charge above Rs 5,000 per day is modelled at 5% GST, subject to its official conditions. A one-day Rs 6,000 private room example therefore totals Rs 6,300. A real hospital must review the contract, item, HSN and current rule before live billing.

The monthly GSTR-1 endpoint produces a review packet with taxable, nil and exempt groupings. It is not an upload-ready government return. The patient rights charter supports the itemised invoice and payment receipt design.

5. Run locally

Docker route

Install Docker Desktop. From the repository root run:

```
docker compose up --build
```

Open <http://127.0.0.1:3000>. The API documentation is at <http://127.0.0.1:8000/docs>. The Compose PostgreSQL port is bound to localhost. The container route is provided for repeatability; native Windows services were used for the tests reported here.

Native Windows route

Install Node.js 24, Python 3.12 and PostgreSQL 16. Create database `tiveri_demo` and a database user that can create tables. In terminal 1, from the repository root:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
$env:DATABASE_URL = 'postgresql://USER:PASSWORD@127.0.0.1:5432/tiveri_demo'
.\.venv\Scripts\python.exe -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000
```

In terminal 2:

```
cd frontend
npm ci
npm run dev
```

To check the stack and code:

```
curl http://127.0.0.1:8000/api/health
.\.venv\Scripts\python.exe -m pytest tests\test_api.py -q
cd frontend
npm run build
```

The database-backed tests cover role access, OPD charges, batch dispensing, GST examples, room stays, TPA, PM-JAY, invoice, payment, refund, tax review and receivables. The doctor to pharmacy to billing handoff was also checked in a live browser.

6. Live demonstration

+ Sign in as Doctor. Select Ananya Rao, save a note with provisional ICD-10, record an investigation and create a prescription.

+ Sign out and enter as Pharmacy. Select the prescription, dispense from a batch and show the stock decrease and GST charge on the running bill.

+ Sign out and enter the Billing desk. Select Ananya and click Generate invoice. The itemised invoice appears below the running bill. Record a sample UPI receipt if desired.

+ For IPD room tax, select Rohan Iyer and add a one-day Rs 6,000 private room stay.

+ For payer paths, select Dev Mehta for simulated TPA preauthorisation or Farah Khan for simulated PM-JAY eligibility and package claim.

Select Reset demo at the Billing desk to restore the original invented cases. It ends the current sessions, so sign in again. The seeded patient history remains available for repeatable practice.

7. Private AWS classroom deployment

This is a deployment plan for invented data. It was not deployed to AWS during development. Keep the application ports private and reach the site by SSH tunnel.

- + Create an Ubuntu EC2 instance. In its security group, allow SSH port 22 from your IP only. Do not expose ports 3000, 8000 or 55433.
- + Install Docker Engine, the Compose plugin and Git using their current official instructions.
- + Clone the repository and start the services with docker compose up --build -d.
- + From the presentation laptop, forward local port 3000 through SSH, then open <http://127.0.0.1:3000>.

```
git clone https://github.com/sreeharix/tiveri-billing.git
cd tiveri-billing
sudo docker compose up --build -d
curl http://127.0.0.1:8000/api/health
```

```
ssh -i KEY.pem -L 3000:127.0.0.1:3000 ubuntu@PUBLIC_IP
```

A real deployment needs stronger authentication, TLS, protected secrets, encryption, backups, monitoring, reviewed tariffs and authorised HIS and payer connectors.

8. References

MoHFW Patients' Rights Charter: clinicalestablishments.mohfw.gov.in/sites/default/files/2023-07/8431.pdf

GST Council 2025 medicine FAQ: gstcouncil.gov.in/sites/default/files/2025-09/faq_0.pdf

CBIC goods and services rates: cbic-gst.gov.in/hindi/gst-goods-services-rates.html

GST Council service rate table, room entry 31A: gstcouncil.gov.in/sites/default/files/2024-02/gstservicerates.pdf

GST Council inpatient AAR: gstcouncil.gov.in, Laxmi Health Care Center order

NHA PM-JAY HBP 2.2 manual: nha.gov.in/img/resources/HBP-2.2-manual.pdf

IRDAI health insurance: irdai.gov.in/health-dept

GST portal GSTR-1 guide: tutorial.gst.gov.in/userguide/returns/GSTR_1.htm

AWS EC2 SSH and security group guides: docs.aws.amazon.com/AWSEC2/

Docker Engine and Compose Ubuntu guides: docs.docker.com/